

Kernel Borderlands (KB)

Behavioral Defense at Ring Zero with Agent-Assisted Orchestration

B.Tech Major Project Synopsis — 2026–2027

Department	Cybersecurity, IoT, and ECE
Institution	Malla Reddy University, Hyderabad
Guide	Mrs. Rashmi, Assistant Professor IV CS Alpha
Batch	IV CS Alpha

Kandukoori Tejaswini	2311CS040077
Kanne Karthik	2311CS040080
Karedla Rupa Yeshvitha	2311CS040082
Konduru Pardhu Sri Rushi Varma	2311CS040089

Abstract

Kernel Borderlands (KB) is a kernel-level runtime defense framework for Linux systems that detects, classifies, and contains anomalous process behavior directly at Ring 0. The system couples a lightweight eBPF observability layer with a graduated enforcement engine and an optional multi-agent decision system (AADS), producing a platform that is simultaneously a production-grade runtime protection tool and a research instrument for studying post-exploitation process behavior under live attack conditions. Unlike signature-based intrusion detection or static policy systems such as SELinux, KB evaluates *behavioral transitions over time* rather than individual system calls, enabling detection of sophisticated multi-stage attacks that appear innocuous at each isolated step.

Problem Statement

Modern Linux systems underpin critical infrastructure ranging from cloud servers to containerized microservices. Existing defense mechanisms exhibit fundamental gaps:

- **Signature-based tools** (antivirus, IDS/IPS) are reactive by design and fail against novel or obfuscated zero-day attacks.
- **Policy-based LSMs** (SELinux, AppArmor) enforce static, administrator-defined rules that cannot adapt when a legitimate binary is hijacked or when an attacker operates entirely within permitted syscalls.
- **Userland monitoring** (audit daemons, SIEM agents) lacks the vantage point required for true behavioral analysis; a sufficiently privileged attacker can silence userland observers without touching the kernel log.

- **Kernel rootkits** operate beneath every userland control plane, manipulating the very data structures defenders rely upon.

KB is designed to address these gaps by observing and enforcing behavior at its source — inside the kernel — without requiring intrusive kernel modifications.

Objectives

1. Instrument Linux kernel execution using eBPF (CO-RE/BTF) to capture syscall distributions, process lineage, privilege changes, memory mapping patterns, file access, and network behavior with sub-millisecond overhead.
2. Implement a per-process Behavioral Scoring Engine (BSE) that computes a continuous risk score using an exponential moving average across six weighted behavioral dimensions.
3. Classify each process into one of three execution zones — Safe, Suspicious, Borderlands — and trigger graduated, reversible containment actions (seccomp filters, namespace isolation, cgroup throttling, or termination).
4. Build an Agent-Assisted Decision System (AADS) comprising Judge, Jury, and Executor agents that augment enforcement decisions through multi-agent coordination without replacing deterministic kernel-layer control.
5. Produce a fully auditable and tamper-evident log chain for every kernel event, score change, zone transition, and enforcement action.
6. Provide a real-time React dashboard and CLI tool (`kbctl`) for operator visibility and manual override.

System Architecture

KB follows a strict four-layer architecture in which each layer communicates only with its immediate neighbors, limiting blast radius in the event of component failure.

Layer 1: eBPF Observability (Ring 0)

eBPF programs are loaded into the kernel and verified by the kernel’s built-in verifier, guaranteeing they cannot crash the system or loop infinitely. Hook points cover: `tracepoint:syscalls` (all syscall entry/exit), `tracepoint:sched` (fork, exec, exit), `kprobe:commit_creds` (privilege changes), `bpf_lsm` (sensitive file access), `tracepoint:net` (network connections), and `kprobe:mmap_region` (memory mapping with executable flags). Captured events are emitted into a perf ring buffer for zero-copy delivery to userland.

Layer 2: Control Plane (Ring 3, Go)

The `kbd` daemon consumes ring buffer events, maintains a per-process state store (SQLite), evaluates YAML-defined operator policies, and exposes a gRPC API through which both hu-

man operators and AADS agents interact with the system. All enforcement actions are authorized and executed exclusively through this layer. Audit logs are written as append-only structured JSON to a tamper-evident chain using SHA-256 linking.

Layer 3: Agent-Assisted Decision System (Python)

The AADS layer is *optional* and *advisory*: no enforcement action may be taken solely on the basis of an agent recommendation without corresponding policy authorization. Three agents collaborate via Apache Kafka:

- **Judge Agent** — consumes the event stream, correlates behavioral patterns across time windows, and generates structured alerts with confidence scores.
- **Jury Agent** — cross-validates Judge alerts against historical process profiles, suppresses known false positives, and produces validated decision records.
- **Executor Agent** — translates validated decisions into Control Plane API calls within its permission scope, monitors outcomes, and records full decision provenance.

Agents are governed by scoped capability tokens. The Judge and Jury have read-only access; only the Executor may submit decisions, and it cannot issue direct kernel operations.

Layer 4: Operator Interface

A React + TypeScript dashboard provides real-time zone distribution, per-process behavioral scores, swarm topology visualization (D3.js), and audit log access over WebSockets. The `kbctl` CLI exposes the full gRPC API for scripted management.

Behavioral Zone Model

The Execution Zone Model is the central abstraction of KB. Every process is continuously assigned to one of three zones based on its smoothed behavioral risk score S_t :

Zone	Score Range	System Response
Safe (Green)	0–39	Passive observation; no enforcement
Suspicious (Yellow)	40–74	Increased monitoring; network syscalls blocked
Borderlands (Red)	75–100	Namespace isolation, cgroup throttle, possible termination

Scores are computed as a weighted composite of six dimensions — syscall entropy deviation (KL divergence, 25%), privilege delta (20%), lineage anomaly (20%), memory behavior (15%), network behavior (10%), and file access (10%) — smoothed via an exponential moving average: $S_t = \alpha s_t + (1 - \alpha)S_{t-1}$, with $\alpha = 0.3$ by default. The key design insight is that *the unit of analysis is the transition between behavioral states, not individual syscalls*: an `execve` following a privilege escalation inside an unexpected process lineage after a burst of `mmap` calls is an attack; the same `execve` in isolation is not.

Technology Stack

Component	Technology
Kernel instrumentation	eBPF (CO-RE/BTF), libbpf, clang/LLVM 12+, Linux 5.8+
Containment primitives	Linux namespaces, seccomp (libseccomp), cgroup v2, <code>prctl()</code>
Control plane	Go 1.21+, gRPC/protobuf, SQLite, PostgreSQL, Cobra CLI
Agent runtime	Python 3.11+, Ray RLLib, asyncio, Gymnasium, PyTorch 2.x
Agent messaging	Apache Kafka 3.x, ZeroMQ, Protocol Buffers v3
Fine-tuning pipeline	Hugging Face Transformers, PEFT (QLoRA), bitsandbytes, TRL
Dashboard	React 18, TypeScript 5, Vite, D3.js, Recharts, WebSockets
Infrastructure	Docker, Docker Compose, Redis, Prometheus, Weights & Biases

Implementation Plan

Foundation Layer: Establish the core eBPF observability layer, Behavioral Scoring Engine, Boundary Classifier, Control Plane daemon, manual containment enforcement, audit logger, CLI tool, and real-time dashboard. All components in this layer are deterministic and carry no AI/ML dependency.

Agent Integration: Layer the AADS system atop the foundation infrastructure, introducing the Judge, Jury, and Executor agents along with multi-agent coordination via Kafka, fine-tuned LLM baselines for heuristic enrichment, automated dataset collection from controlled attack simulations (Metasploit, ADFA-LD, BETH datasets), and a graphical policy authoring tool.

Expected Outcomes

- A working eBPF-based runtime defense system demonstrating real-time behavioral anomaly detection with measurable performance overhead below 3% on standard server workloads.
- Containment action latency under 5 ms for namespace isolation and under 1 ms for seccomp and cgroup enforcement.
- A labeled syscall-trace dataset generated through controlled attack simulations, suitable for ML baseline training and future research use.
- A multi-agent coordination framework in which AI-driven recommendations are auditable, constrained by policy, and overridable by human operators.
- Comprehensive audit logs, operator dashboard, and CLI tooling constituting a deployable research platform for Linux kernel-level behavioral security.

Conclusion

Kernel Borderlands addresses a genuine gap in Linux runtime security: the inability of existing tools to detect and contain sophisticated, multi-stage behavioral attacks at the kernel level. By coupling lightweight eBPF instrumentation with a graduated enforcement ladder and an advisory multi-agent system, KB achieves deep kernel visibility with operational safety — keeping humans in control, AI in a supporting role, and the kernel clean of invasive modifications. The project simultaneously delivers a production-capable security framework and a research platform for studying advanced attack behaviors at Ring 0.